Course: DSA with C++

Introduction

Sorting is the process of arranging elements in a particular order, most commonly either in ascending or descending order.

It is one of the most frequently carried out tasks in computer programming and used in:

- **Data Analysis:** To organize and analyze large datasets.
- **User Interfaces:** To display items in systematic order, such as sorting files by name, date, or size.
- **Database Systems:** To order records and perform faster search operations.
- **Algorithms Design:** Some algorithms rely on input data being sorted.

Different Sorting Algorithms

In this chapter, we will learn about the internal workings of the following sorting algorithms:

- Bubble Sort
- Selection Sort
- Insertion Sort
- Merge Sort
- Quick Sort
- Counting Sort

Each algorithm offers different levels of efficiency, depending on the dataset and its use case.

Let's start with bubble sort.

Introduction

Bubble sort works by repeatedly comparing two adjacent elements and swapping them if they are not in the correct order.

Here's how the algorithm works:

- Iterate through the vector, comparing each pair of adjacent elements.
- Swap the elements if the preceding element is greater than the next element.
- Repeat this process for each element until the vector is sorted.

Next, we will learn the workings of bubble sort in detail.

Working of Bubble Sort

Suppose we need to sort a vector of numbers in ascending order.

Figure: Unsorted Vector

Bubble sort works by comparing two adjacent elements and swapping them if needed.

Figure: Compare and Swap if Needed

So, we compare and swap (only if needed) all elements with their next element:

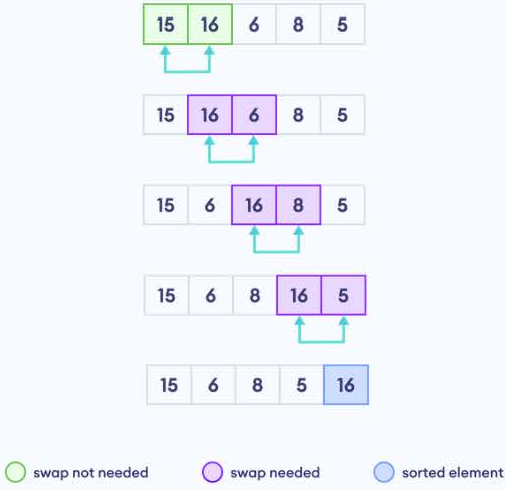

Figure: Compare Adjacent Elements

After these comparisons, the largest element, **16**, will be placed at the last position.

As we can see, at the end of the iteration, the largest element is where it needs to be – the last index.

However, all elements before the last index are unsorted. So, now we will now repeat the same process for the unsorted elements.

Working of Bubble Sort

Let's repeat the process again up to the second-last position.

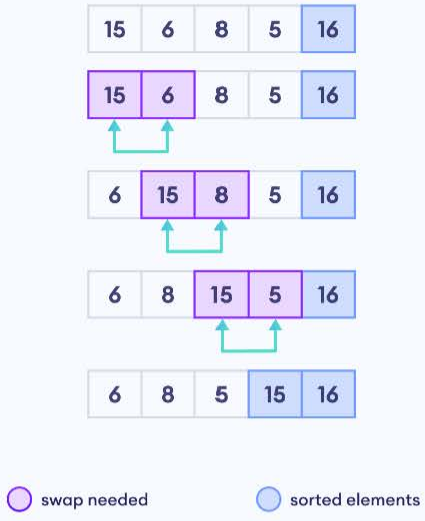

Figure: Compare Adjacent Elements

Now, **15** (the second-largest element) is placed in the second-last position.

Note: Here, we don't compare the second-last element **15** with the last element **16** because **16** is already in its sorted position.

Now the sorted part includes the second largest and largest element in the second last and the last position respectively.

As can be seen at the end of every iteration, only the largest element of the unsorted part is sorted while the rest of the vector remains unsorted.

That is why, to get a sorted vector, we have to compare each element with every other element in the vector.

So, we'll repeat this process for the rest of the unsorted elements.

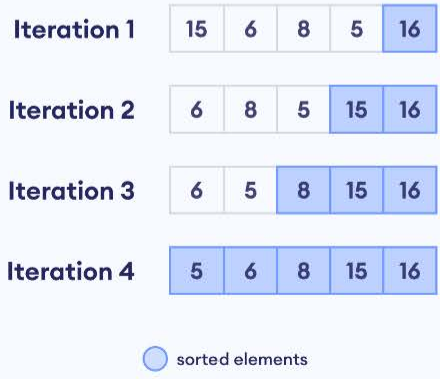

Figure: Iterations in Bubble Sort

This is how bubble sort arranges all elements in a vector.

Working of Bubble Sort

Let's repeat the process again up to the second-last position.

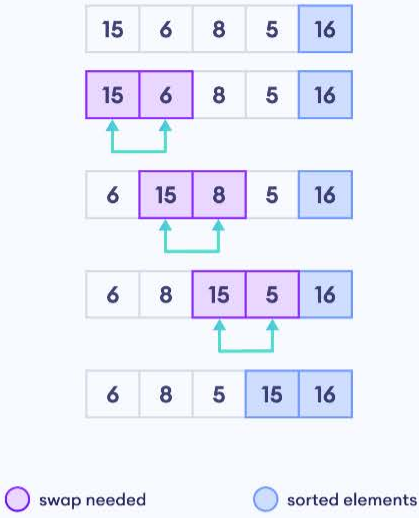

Figure: Compare Adjacent Elements

Now, **15** (the second-largest element) is placed in the second-last position.

Note: Here, we don't compare the second-last element **15** with the last element **16** because **16** is already in its sorted position.

Now the sorted part includes the second largest and largest element in the second last and the last position respectively.

As can be seen at the end of every iteration, only the largest element of the unsorted part is sorted while the rest of the vector remains unsorted.

That is why, to get a sorted vector, we have to compare each element with every other element in the vector.

So, we'll repeat this process for the rest of the unsorted elements.

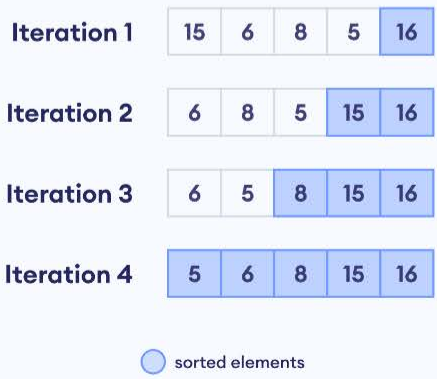

Figure: Iterations in Bubble Sort

This is how bubble sort arranges all elements in a vector.

Thought Process to Implement Bubble Sort

Now that we understand the workings of bubble sort, let's see how we can implement the programming logic based on its principles.

1. Compare the first element with its next element and swap them if necessary.

Bubble Sort is based on comparing and swapping two adjacent elements, so we need to write code that does the same.

```
if (arr[0] > [arr[1]) {
    // Swap adjacent elements
    swap(arr[0], arr[1];
}
```

Based on the order in which we are sorting, we might have to change the operation

- `arr[0] > arr[1]` for ascending order
- `arr[0] < arr[1]` for descending order

2. Perform comparison and swapping for all vector elements

Our algorithm compares each element with its next element, so we will use a for loop.

```
for (int j = 0; j < arr.size() - 1; ++j) {
    // Compare and swap if they are in wrong order
    if (arr[j] > arr[j+1]) {
        swap(arr[j], arr[j+1]);
    }
}
```

Since the last element doesn't have any next element, we only need to run the loop up to the last element (denoted by `j < arr.size() - 1`).

This will sort the largest element in the vector.

Thought Process to Implement Bubble Sort

3. Sort the vector for the rest of the elements.

Once the largest element is sorted, we have to restart the comparison for the remaining unsorted elements.

For this, we can use another for loop.

```
for(int i = 0; i < arr.size(); ++i) {
    for(int j = 0; j < arr.size() - 1; ++j) {
        // Swap adjacent elements if they are in the wrong order
        if(arr[j] > arr[j+1]) {
            swap(arr[j], arr[j+1]);
        }
    }
}
```

Once the inner loop completes, the largest element from the unsorted part will move to the sorted part.

The outer loop then restarts the comparison from the first element.

4. Remove redundant checks.

With each iteration of the outer loop, one element will settle in its correct position, starting from the largest element.

Figure: Demonstration of Sorted Portion

However, our inner loop always iterates up to the second-last element, making comparisons even for elements that are already in their correct positions.

```
for(int j = 0; j < arr.size() - 1; ++j)
```

To avoid the comparison of sorted elements, we can modify our code to iterate one time less each iteration.

```
for(int j = 0; j < arr.size() - i - 1; ++j)
```

Similarly, when our vector is sorted from the last to the second position, the first element will be automatically sorted.

So, we can decrease the iteration of the outer loop by 1 (i < arr.size() - 1).

```
for(int i = 0; i < arr.size() - 1; ++i)
```

Thought Process to Implement Bubble Sort

5. Terminate early if no more modifications are necessary

In bubble sort, each iteration of the outer loop places one element in its correct position.

However, if no swaps occur during an entire pass of the inner loop, we know that elements are already in their correct positions–the vector is sorted.

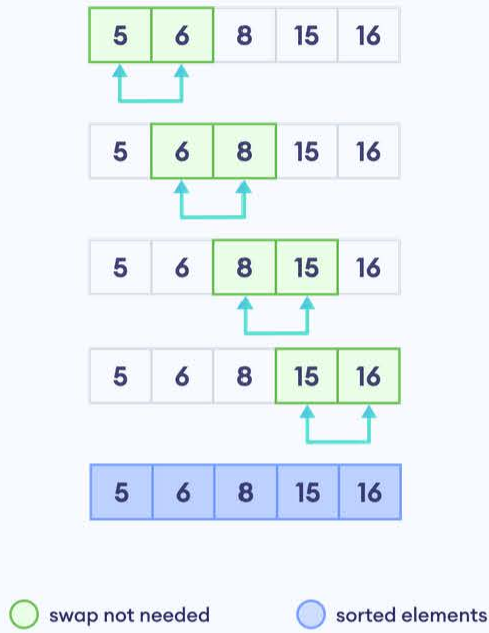

Figure: No Swaps Occur in a Sorted Vector

So, let's add a flag to detect if a swap occurred during an iteration.

```
bool modified = false;
```

We'll set the flag to true if any swaps occur during the process.

```
if(arr[j] > arr[j+1]) {
    swap(arr[j], arr[j+1]);
    // Set modified to true indicating a swap occurred
    modified = true;
}
```

However, if no swaps occur, we'll terminate the process immediately.

```
// If no elements were swapped,
// the arr is sorted and we can break early
if(!modified)
    break;
```

Let's now create a function `bubble_sort()` to wrap all our logic.

```
void bubble_sort(vector<T>& arr) {
}
```

Source Code: Bubble Sort

```cpp
// Template to perform bubble sort on a vector
template<typename T>
void bubble_sort(vector<T>& arr) {

    // Flag to detect if any swapping happened in the current pass
    bool modified = false;

    for(int i = 0; i < arr.size() - 1; ++i) {
        // Reset modified to false at the start of each pass
        modified = false;

        // Stop at arr.size()-1-i to avoid checking sorted elements
        for(int j = 0; j < arr.size() - 1 - i; ++j) {
            // Swap adjacent elements if they are in the wrong order
            if(arr[j] > arr[j+1]) {
                swap(arr[j], arr[j+1]);
                // Set modified to true indicating a swap occurred
                modified = true;
            }
        }

        // If no elements were swapped,
        // the arr is sorted and we can break early
        if(!modified)
            break;
    }
}
```

Run Code >>

Output

```
Unsorted Vector: 15 16 6 8 5
Sorted Vector: 5 6 8 15 16
```

Practice:

Bubble Sort

Easy

New to Practice? Take a tour

Problem Description

Can you write the bubble sort program on your own?

- Create a function named `bubble_sort()` that takes a vector as its argument.
- Sort the vector in ascending order within the function.
- Print the sorted vector from outside the function.

Example

```
Test Input
1 15 6 8 2 5 9
```

```
Expected Output
1 2 5 6 8 9 15
```

Was this helpful? Yes No

Quiz: In bubble sort, what happens if no swaps are made in an iteration?

A: The algorithm continues to the next iteration.

B: The algorithm sorts the rest of the elements.

C: The algorithm terminates.

D: The algorithm starts over.

Quiz: In bubble sort, what happens if no swaps are made in an iteration?

A: The algorithm continues to the next iteration.

B: The algorithm sorts the rest of the elements.

C: The algorithm terminates.

Correct Answer!

The algorithm recognizes the vector is sorted and terminates early.

D: The algorithm starts over.

Was this helpful? Yes No

Practice:

Bubble Sort

Easy

New to Practice? Take a tour

Problem Description

Write a program to sort the vector elements in descending order using bubble sort.

- Create a function named `bubble_sort()` that takes a vector as an argument.
- Inside the function, sort the vector in descending order using bubble sort.
- Print the sorted vector outside the function.

Example

```
Test Input
1 15 6 8 2 5 9
```

```
Expected Output
15 9 8 6 5 2 1
```

Was this helpful? Yes No

Complexity Analysis

The table below summarizes the time and space complexities of bubble sort:

Best Case Time Complexity	$O(n)$
Worst Case Time Complexity	$O(n^2)$
Average Case Time Complexity	$O(n^2)$
Space Complexity	$O(1)$

To learn more about the complexity and applications of bubble sort, visit this blog.

Introduction

Selection sort works by repeatedly selecting the smallest (or largest) unsorted element and placing it towards the beginning of the vector.

Here's how the algorithm works:

- Select the smallest element from the vector.
- Swap this element with the first element.

At this point, the correct element is placed at the first position. However, the portion of the vector from the second position to the last remains unsorted. Therefore,we will repeat this process:

- Select the smallest element from the remaining unsorted part (from the second to last positions) of the vector.
- Swap this element with the second element.
- Continue this process for each following position until the entire vector is sorted.

Next, we will understand the working with the help of images.

Working of Selection Sort

Suppose we have the following unsorted vector.

Figure: Unsorted Vector

Let's implement selection sort to sort this vector in ascending order.

1. Find the smallest element and swap it with the first element in the vector.

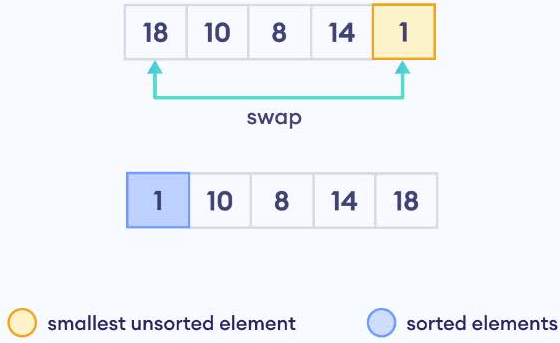

Figure: Swap Smallest Element and First Element

At this point, the first position includes the smallest element. However, the vector from the second position to the last remains unsorted.

2. Find the smallest element in the unsorted portion of the vector and swap it with the second element.

2. Find the smallest element in the unsorted portion of the vector and swap it with the second element.

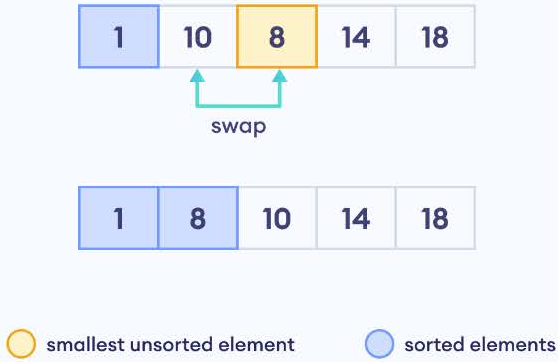

Figure: Swap Element at the Second Position

3. Repeat this process until all the elements are sorted.

The image below shows the final step in this process.

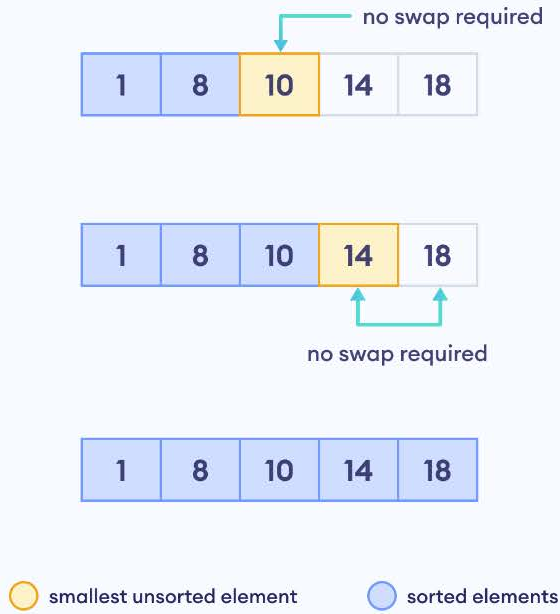

Figure: Sorted Vector

Quiz: Suppose we are sorting elements in ascending order using selection sort. After sorting the smallest element, what is the next action in the selection sort?

A: Re-sort the entire vector.

B: Find the second-smallest element and swap it with the element at the first position.

C: Find the smallest element and swap it with the element at the second position.

D: Find the second-smallest element and swap it with the element at the second position.

Quiz: Suppose we are sorting elements in ascending order using selection sort. After sorting the smallest element, what is the next action in the selection sort?

A: Re-sort the entire vector.

B: Find the second-smallest element and swap it with the element at the first position.

C: Find the smallest element and swap it with the element at the second position.

D: Find the second-smallest element and swap it with the element at the second position.

Correct Answer!

The second-smallest element should be swapped with the element in the second position.

Quiz: Given the following vector of integers:

```
{5, 2, 9, 1, 5, 6}
```

What will the vector look like after one iteration of selection sort in ascending order?

A: `{1, 2, 9, 5, 5, 6}`

B: `{1, 2, 5, 5, 6, 9}`

C: `{5, 9, 2, 5, 1, 6}`

D: `{5, 2, 9, 1, 5, 6}`

Quiz: Given the following vector of integers:

```
{5, 2, 9, 1, 5, 6}
```

What will the vector look like after one iteration of selection sort in ascending order?

A: `{1, 2, 9, 5, 5, 6}`

B: `{1, 2, 5, 5, 6, 9}`

C: `{5, 9, 2, 5, 1, 6}`

D: `{5, 2, 9, 1, 5, 6}`

Correct Answer!

The smallest element **1** is swapped with the first element **5**.

Was this helpful? Yes No

Thought Process to Implement Selection Sort

Consider the vector we introduced in the previous section.

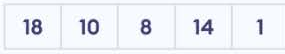

Figure: Unsorted Vector

The first step of selection sort involves finding the smallest element in the vector.

1. Find the index of the smallest element in the vector.

We initially assume that the first element is the smallest element.

```
// Index of the first element
int min_idx = 0;
```

Then we iterate through the vector starting from the second element (index 1), and compare each element with the current minimum element.

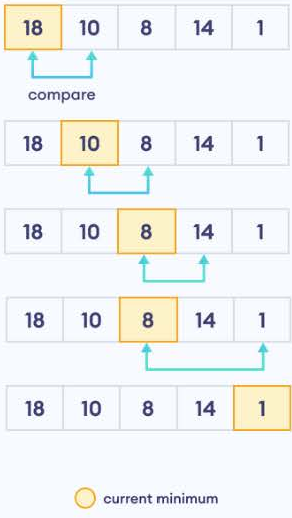

Figure: Find the Smallest Element

If we find a smaller element, we update the `min_idx` with the corresponding index.

```
// Iterate over all the elements starting at index 1
for(int j =  1; j < arr.size(); ++j) {
    // Update if a smaller element is found
    if(arr[j] < arr[min_idx])
        min_idx = j;
}
```

2. Swap the smallest element with the first element.

After finding the smallest element from the loop, we will swap that element with the first element.

```
swap(arr[0], arr[min_idx]);
```

Thought Process to Implement Selection Sort

3. Repeat steps 1 and 2 for all the other elements in the list.

At this point, the smallest element is placed at the first position. We have to repeat the same process of finding the smallest element and swapping for all the remaining elements.

Let's use another for loop to ensure all the elements of the vector are considered.

```
for(int i = 0; i < arr.size(); ++i) {
    // Assume the first unsorted element is the minimum
    int min_idx = i;

    // Iterate over unsorted elements
    for(int j =  i + 1; j < arr.size(); ++j) {
        // Find index of the smallest element
        if(arr[j] < arr[min_idx])
            min_idx = j;
    }

    // Swap the smallest element with
    // the first element of the unsorted part
    swap(arr[i], arr[min_idx]);
}
```

Here's how the above code works:

1. In the first iteration of the outer loop, we identify the smallest element of the unsorted vector and swap it with the first element.

The structure of the vector at this point will be the smallest element at the first position and the unsorted vector from the second element.

1. In the second iteration, we identify the smallest element from the unsorted portion of the vector and swap it with the second element.

The process continues until the entire vector is sorted.

Now, let's use a function `selection_sort()` to wrap our logic.

```
void selection_sort(vector<T>& arr) {
}
```

Source Code: Selection Sort

```cpp
// Template to perform selection sort
template<typename T>
void selection_sort(vector<T>& arr) {

    for(int i = 0; i < arr.size(); ++i) {
        // Assume the first unsorted element is the minimum
        int min_idx = i;

        // Iterate over unsorted elements
        for(int j =  i + 1; j < arr.size(); ++j) {
            // Find index of the smallest element
            if(arr[j] < arr[min_idx])
                min_idx = j;
        }

        // Swap the smallest element with
        // the first element of the unsorted part
        swap(arr[i], arr[min_idx]);
    }
}
```

Run Code >>

Output

```
Unsorted Vector: 18 10 8 14 1
Sorted Vector: 1 8 10 14 18
```

Course: DSA with C++

Practice:

Selection Sort

Easy

New to Practice? Take a tour

Problem Description

Can you write the selection sort program on your own?

- Create a function named `selection_sort()` that takes a vector as its argument.
- Sort the vector in ascending order within the function and return the sorted vector.
- Print the sorted vector outside the function.

Example

```
Test Input
1 15 6 8 2 5 9
```

```
Expected Output
1 2 5 6 8 9 15
```

Was this helpful? Yes No

Complexity Analysis

Here's the time and space complexities of selection sort:

Time Complexity	**$O(n^2)$**
Space Complexity	`O(1)`

Selection sort has a time complexity of $O(n^2)$ in all cases.

To learn more about the complexity and applications of selection sort, visit this blog.

Introduction

Insertion sort is an efficient sorting algorithm that builds a sorted vector one element at a time, gradually creating a larger and larger sorted portion of the vector.

Here's how the algorithm works:

- Take an element from the unsorted portion of a vector
- Inserting it into its correct position within the already-sorted portion

Understanding insertion sort solely from its definition can be difficult.

So, let's visualize the workings of insertion sort using images.

Working of Insertion Sort

Suppose we need to sort the vector below:

Figure: Unsorted vector

To sort this vector in ascending order, we follow these steps:

1. Divide the vector into sorted and unsorted portions.

Initially, we assume the first element is sorted, and the rest of the vector is unsorted.

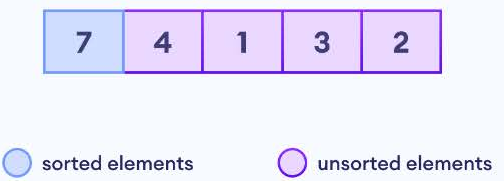

Figure: Sorted and Unsorted Portions

2. Set the first element of the unsorted portion as the key element.

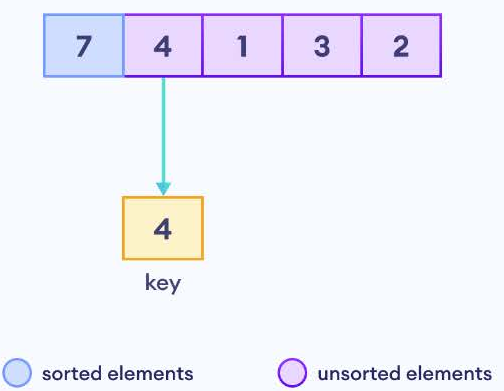

Figure: Determine the Key

Working of Insertion Sort

3. Insert key into its correct position.

We will compare the key element with elements to its left (sorted portion).

- If elements in the sorted portion are larger than the key, we will shift them to the right.
- We will then insert the key element in its position.

For example, our key element here is **4**. So, we'll compare the value of the key element to the element to its left i.e **7**.

4 is less than **7**, so we shift the sorted element, **7**, to the right.

Then, we Insert the key element, **4**, in its position.

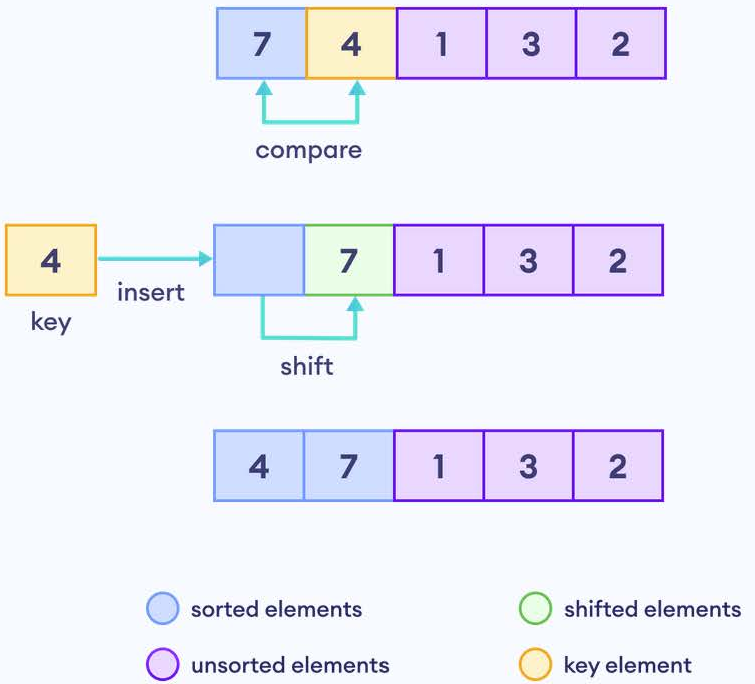

Figure: Insertion in the First Iteration

Note: Instead of swapping, we insert the key element into its correct position. That's why the algorithm is called insertion sort.

Note: Instead of swapping, we insert the key element into its correct position. That's why the algorithm is called insertion sort.

4. Repeat Step 3 for all the elements of the unsorted part.

The new key element is **1** (first element of the unsorted part).

We'll compare the **key** element with the **elements to its left one by one.**

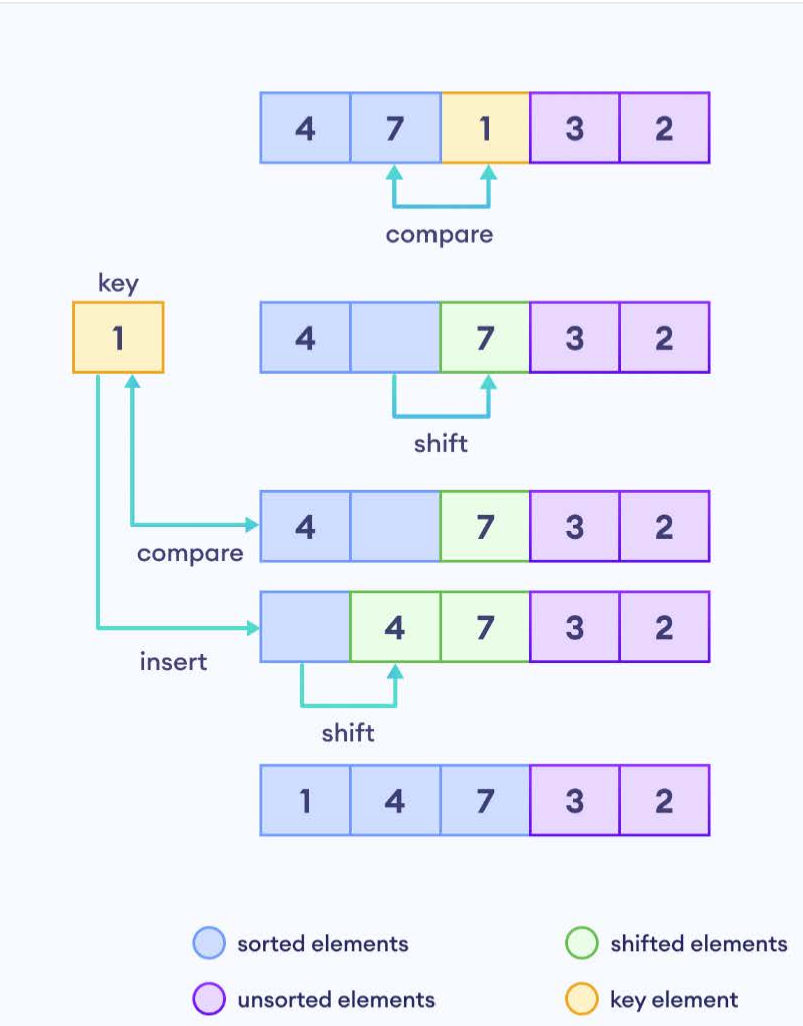

Figure: Insertion in the Second Iteration

Here's how to interpret this image:

1. Since **7** is larger than the key element, it is shifted to the right.
2. Again, **1** is compared to **4** and since **4** is larger, we shift **4** to right as well.
3. Finally, **1** is inserted at the first position.

Note: If elements in the sorted portion are smaller or equal to the key, we stop the comparison because the element is already in the correct position.

Working of Insertion Sort

Here's how the rest of the insertion sort works:

Since **3** is our new key element, we compare it with the elements to its left starting with **7**.

We'll then insert it in its correct position which is after **1**.

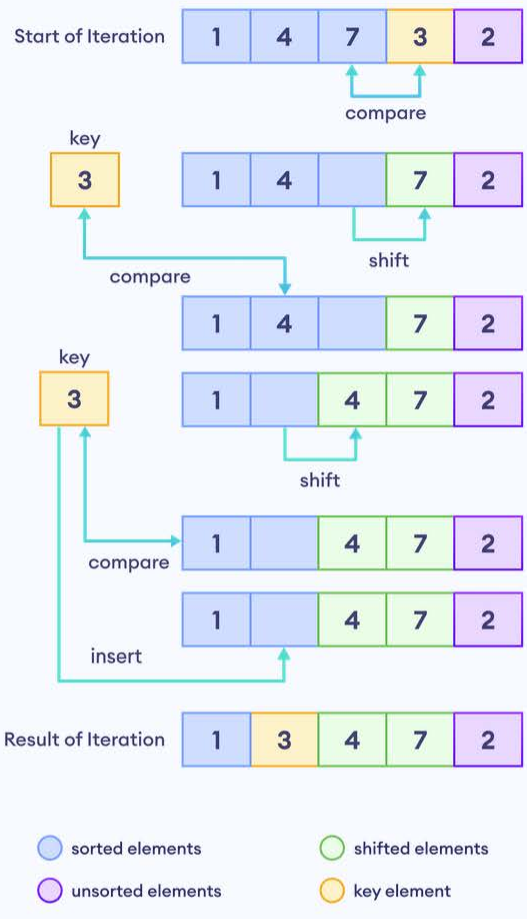

Figure: Insert 3 in the Correct Position

Finally, we'll take **2** as the key element.

We shift elements **3**, **4**, and **7** as they are larger than our key and insert the element **2** in second position.

Figure: Insert 2 in the Correct Position

That's how we sort a vector using insertion sort.

Quiz: Which of the following statements best describes the insertion sort algorithm?

A: It repeatedly swaps adjacent elements if they are in the wrong order.

B: It repeatedly picks the smallest element from the unsorted portion and places it in its correct sorted position.

C: It divides the vector into a sorted and an unsorted portion. It then repeatedly takes the first element from the unsorted portion and inserts it into its correct position within the sorted portion.

D: None of the above.

Quiz: Which of the following statements best describes the insertion sort algorithm?

A: It repeatedly swaps adjacent elements if they are in the wrong order.

B: It repeatedly picks the smallest element from the unsorted portion and places it in its correct sorted position.

C: It divides the vector into a sorted and an unsorted portion. It then repeatedly takes the first element from the unsorted portion and inserts it into its correct position within the sorted portion.

Correct Answer!

This accurately describes the Insertion Sort algorithm.

D: None of the above.

Quiz: Suppose we are using insertion sort to sort elements in ascending order.

When do we stop the comparison for the key element (the first element of the unsorted part)?

A: When the key element is smaller than the element to its right.

B: When the key element is larger than or equal to the element to its left.

C: When the key element is the smallest in the vector.

D: When the key element is at the end of the vector.

Quiz: Suppose we are using insertion sort to sort elements in ascending order.

When do we stop the comparison for the key element (the first element of the unsorted part)?

A: When the key element is smaller than the element to its right.

B: When the key element is larger than or equal to the element to its left.

Correct Answer!

This is the correct stopping condition for the key element during insertion sort.

C: When the key element is the smallest in the vector.

D: When the key element is at the end of the vector.

Thought Process to Implement Insertion Sort

Let's see how we can implement insertion sort in C++.

1. Select the key element.

We know that the `key` will always be the first element of the unsorted portion.

So if we take the second element (index **1**) as the key, it will virtually divide the vector into two portions:

- Sorted portion containing element at index less than **1**.
- Unsorted portion containing elements since index **1**.

```
T key = arr[1];
```

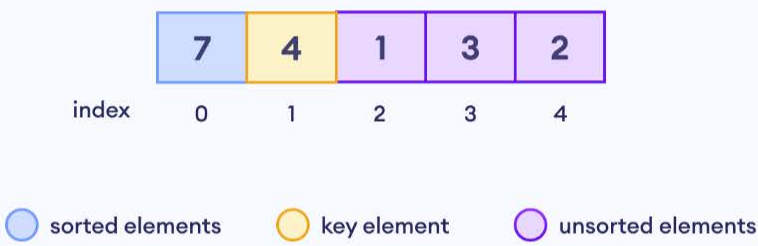

Figure: Sorted and Unsorted Parts

2. Compare the key with the sorted elements one by one (from right to left) until the key element is either equal to or smaller.

At this point, we know the index of the key element (key index, `i = 1`).

Now we need to compare the key to element on the left. Meaning, key `arr[i]` will be compared with elements from `arr[i - 1]`.

If the element at index `i - 1` is smaller than the key at index `i`, we move the element rightwards.

```
while (key < arr[i - 1]) {
    // Shift element rightward
    arr[i] = arr[i - 1];
}
```

Then we insert the key in its appropriate place.

```
arr[i - 1] = key;
```

Next, we will see how we can use a loop to repeat this process for all the elements in the list.

Thought Process to Implement Insertion Sort

3. Find the next key element

The previous iteration inserts the key element at its right position.

Now, we need to find the new key element, which will be the next element after the previous key `(i + 1)`. For this, we can use a for loop.

```
for (int i = 1; i < arr.size(); ++i)
```

4. Repeat Step 2 and 3 until all elements are sorted.

We will again compare the key with elements in the left (sorted portion) and shift elements accordingly.

We can use a new variable `j` to access elements in the sorted portion.

```
int j = i - 1;
```

We will use a while loop to compare the key with every element in the sorted portion and shift elements rightward if they are greater than the key.

```
while (key < arr[j]) {
    // Shift element rightward
    arr[j+1] = arr[j]
    --j;
}
```

And finally place the key in its appropriate position.

```
arr[j+1] = key;
```

Here, the value of `j` starts from `i - 1`, so we have to add a condition `j >= 0` to prevent it from going out of bounds of the vector.

```
while(j >= 0 && key < arr[j]){
    // Shift element rightward
    arr[j+1] = arr[j]
    --j;
}
```

Finally, we will use a function `insertion_sort()` to wrap all our logic.

```
void insertion_sort(vector<T>& arr) {

}
```

Source Code: Insertion Sort

```cpp
// Template to perform insertion sort on a vector.
template<typename T>
void insertion_sort(vector<T>& arr) {
    for(int i = 1; i < arr.size(); ++i) {
        // Start with the second element and
        // insert it where it belongs in the sorted left part.
        T key = arr[i];
        int j = i - 1;

        // Move elements greater than key
        // one position ahead of their current position
        while(j >= 0 && key < arr[j]) {
            // Shift element rightward
            arr[j + 1] = arr[j];
            --j;
        }
        // Place key in its correct position
        arr[j + 1] = key;
    }
}
```

Run Code >>

Output

```
Unsorted Vector: 7 4 1 3 2
Sorted Vector: 1 2 3 4 7
```

Complexity Analysis

The various time and space complexities of insertion sort is given below:

Best Case Time Complexity	`O(n)`
Worst Case Time Complexity	$O(n^2)$
Average Case Time Complexity	$O(n^2)$
Space Complexity	`O(1)`

To learn more about the complexity and applications of insertion sort, visit this blog.